

课程笔记

[CS25] Mixture of Experts / Switch Transformer —Barret Zoph, Google

基于公开课程资料整理

2021 年 9 月

Preventing Dropping with No-Token-Left-Behind

An attempt to avoid dropped tokens by sending to the *next-best* expert (if the preferred expert is at capacity).

Empirically this approach doesn't work that well.

Token wants to get routed to its highest probability expert.

Router Probabilities
0.1 0.7 0.5 0.9 0.3 0.7
0.7 0.2 0.3 0.1 0.1 0.1
0.2 0.1 0.2 0.1 0.8 0.2

Expert 1 Expert 2 Expert 3

Stage-2
Route token to second highest probability if not routed

Stage-1
Route token to highest probability

Tokens

Google

Stanford

Barret Zoph

视频作者/频道: Stanford CS25

发布日期: 2021

视频时长: 约 60 分钟

视频链接: <https://www.youtube.com/watch?v=Q5baLehv5So>

项目主页: <https://github.com/hqhq1025/ai-course-notes>

目录

1 引言：通过稀疏性扩展 Transformer	4
1.1 本章小结	4
2 Mixture of Experts (MoE) 基础	4
2.1 MoE 架构	4
2.2 本章小结	4
3 MoE 发展脉络	4
3.1 从早期混合专家到深度学习 MoE	4
3.2 以 timeline 观察进化	5
3.3 本章小结	6
4 Switch Transformer：简化 MoE	6
4.1 核心创新：Top-1 路由	6
4.2 容量因子 (Capacity Factor)	6
4.3 负载均衡损失	6
4.4 本章小结	6
5 关键帧与幻灯片图解	7
5.1 路由关键帧	7
5.2 训练与推理堆栈	7
5.3 本章小结	8
6 专家负载与调度治理	8
6.1 负载均衡信号	8
6.2 失败模式演练	8
6.3 本章小结	9
7 逐段精读：Barret 现场讲解	9
7.1 开场 (00:04–05:00)：为什么要加稀疏性	9
7.2 路由细节 (05:00–25:00)：Router 的挑战	9
7.3 部署与结尾 (45:00–60:00)：案例与未来	9
7.4 本章小结	10
8 实验结果	10
8.1 预训练加速	10
8.2 Top-1 vs. Top-2 路由对比	10
8.3 知识蒸馏	10
8.4 本章小结	10
9 路由器与调度实践	10
9.1 Router 训练：信号与负载	10
9.2 调度与推理吞吐	11
9.3 本章小结	11

10 Switch Transformer 操作手册	11
10.1 训练配置 checklist	11
10.2 指标与监控 playbook	12
10.3 本章小结	12
11 音频/字幕补充	12
11.1 Transcription replay	12
11.2 本章小结	12
12 时间线与故障演练	12
12.1 完整时间线	12
12.2 故障演练流程	13
12.3 本章小结	13
13 MoE 策略与控制要点	13
13.1 Control loop overview	13
13.2 治理与保护层	14
13.3 本章小结	14
14 Extended Transcript Narrative	14
14.1 00:00–10:00: Scaling Motivation	14
14.2 10:00–20:00: Router Designs	14
14.3 20:00–40:00: Training Infrastructure	14
14.4 40:00–60:00: Deployment stories	14
14.5 本章小结	15
15 Implementation QA Checklist	15
15.1 Pre-training stage	15
15.2 Fine-tuning / distillation stage	15
15.3 Serving stage	15
15.4 本章小结	15
16 Glossary Deep Dive	15
16.1 本章小结	16
17 案例与治理	16
17.1 Case Study: Switch Transformer vs GShard	16
17.2 MoE Governance Checklist	17
17.3 Case Study: Mixtral & Mistral	17
17.4 本章小结	18
18 术语与概念速查	18
18.1 Glossary	18
18.2 本章小结	18

19 部署与成本视角	18
19.1 推理策略与 Expert fusion	18
19.2 运维与监控	19
19.3 本章小结	19
20 路由数学与失败模式	19
20.1 Router 数学细节	19
20.2 失败模式与可观测信号	19
20.3 本章小结	20
21 训练与服务堆栈	20
21.1 训练堆栈：从 optimizer 到 checkpoint	20
21.2 Serving 堆栈：Inference 设备分布	20
21.3 本章小结	21
22 推理成本与基准解释	21
22.1 FLOPs vs latency tradeoffs	21
22.2 Benchmark 解释	22
22.3 本章小结	22
23 展望与讨论	22
23.1 稀疏性的未来	22
23.2 案例研究：GShard 与 GLaM	23
23.3 MoE 部署成熟度	23
23.4 本章小结	23
24 总结与延伸	23
24.1 总结表	24
24.2 延伸阅读	24

1 引言：通过稀疏性扩展 Transformer

Barret Zoph（与 Erwin 联合演讲）介绍了通过**稀疏性（Sparsity）**来扩展 Transformer 的研究。核心动机来自缩放定律（Scaling Laws）——模型性能随规模（计算、数据、参数）的增长而可预测地提升。

1.1 本章小结

本节说明稀疏性背后的 scaling logic：通过选择性激活降低 per-token FLOPs 但提升模型容量。讲者直接把密集模型和 MoE 模型的计算曲线进行了对比——引导我们带着“参数 vs FLOPs”的问题进入后续讲解。

稀疏模型的核心思想

传统密集模型对每个输入使用所有参数。稀疏模型（Mixture of Experts, MoE）让**不同的输入激活不同的参数子集**，从而在不成比例增加计算量的情况下大幅增加模型参数量。

2 Mixture of Experts (MoE) 基础

2.1 MoE 架构

MoE 层替换 Transformer 中的标准 FFN 层：

- 包含 N 个独立的“专家”（每个是一个 FFN）
- 一个**路由网络（Router/Gate）**决定每个 token 发送给哪些专家
- 标准方案：Top- k 路由——每个 token 选择得分最高的 k 个专家

$$y = \sum_{i=1}^N g_i(x) \cdot E_i(x), \quad g(x) = \text{TopK}(\text{softmax}(W_g \cdot x))$$

MoE 的历史

MoE 的思想可以追溯到 1991 年 Jacobs 等人的工作。在深度学习时代，Shazeer et al. (2017) 首次将 MoE 成功应用于大规模 LSTM 语言模型。Switch Transformer 进一步简化了 MoE 并将其应用于 Transformer。

2.2 本章小结

MoE 通过条件计算实现了参数量和计算量的解耦——模型可以拥有万亿参数，但每个 token 只使用其中一小部分。

3 MoE 发展脉络

3.1 从早期混合专家到深度学习 MoE

- 1991 年 Jacobs 等人提出“experts + gating”，初步探索局部模型的组合

- 2017 年 Shazeer et al. 在 LSTM 上展示 Sparsely-Gated MoE，开启工程化大规模 MoE 的时代
- 2021 年 GShard 将 MoE 与 Transformer 结合，利用设备感知分片提高吞吐
- Switch Transformer 进一步简化路由，成为稀疏 Transformer 的开山之作

稀疏模型的关键判据

从历史上看，直到 routers 能维持较低的 expert skew，MoE 才具备可训练性。每一次迭代都在 router、load balancing 和 expert-count 三个维度做技术积累。Switch Transformer 的贡献在于把这些维度封装在一套可控流水线。

3.2 以 timeline 观察进化

- 2021 年早春：GShard 在多个 data-center 上训练 600B 模型，展示 MoE 的规模能力
- 同年末：Switch Transformer 带来 Top-1 router 以降低通信复杂度
- 2022-2024：GLaM、Grok、Mixtral 等模型把 mixture of sizes + distillation 实践到生产线
- 2025 年：Barret 在 CS25 提到“稀疏性要软着陆，需要监控 + governance”，暗示下一阶段是闭环工程

表 1: MoE 重要里程碑及其工程亮点

时期	里程碑	工程聚焦
1991–2015	专家组合与 gating 理论	理论分析、EM 训练、局部专家
2017	Sparsely-Gated MoE (Shazeer et al.)	扩展到大规模 LSTM、引入 router logits
2021	GShard/Switch (Transformer)	device-aware routing、Top-1 路由、load loss
2022–2024	GLaM、Mixtral、Grok	size mix、distillation pipeline、multi-tier routing
2025	工程治理闭环	Token overflow monitoring、expert health logging

演化的治理信号

每次 MoE 升级都会引入新的 observability：从 early gating statistics 到 GShard 的 per-device routing map，再到 Barret 所说的“router health audit”。治理的成熟度决定了稀疏模型能否在 production 里长时间跑通。

Barret 在 2025 CS25 中反复提到“monitoring surfaces”——这些 surface 其实对应表中每一行的工程聚焦。我们也因此把 timeline 和训练/服务堆栈图结合起来，为后续治理章节提供背景。

3.3 本章小结

MoE 发展历程是由路由 → 通信 → 治理三条线构成。每一代模型都在让稀疏计算更可控、更容易监测，这也是 Switch Transformer 能大放异彩的根本原因。

4 Switch Transformer: 简化 MoE

4.1 核心创新: Top-1 路由

Switch Transformer 的简化

Switch Transformer 将路由简化为 **Top-1**——每个 token 只路由到一个专家。这带来了：

- 更低的计算开销（每个 token 只经过一个专家 FFN）
- 更简单的实现和通信模式
- 在保持质量的同时获得更高的训练吞吐量

4.2 容量因子 (Capacity Factor)

每个专家有一个固定大小的缓冲区来接收 token：

$$\text{专家缓冲区大小} = \text{Capacity Factor} \times \frac{\text{总 token 数}}{\text{专家数}}$$

- Capacity Factor = 1.0: 理想均匀分配
- Capacity Factor > 1.0: 允许负载不均衡，减少 token 溢出
- 溢出的 token 跳过 MoE 层，仅通过残差连接

4.3 负载均衡损失

专家坍塌问题

如果没有额外约束，路由器可能将所有 token 都发送给少数几个专家，导致其余专家未被训练——这就是**专家坍塌**。Switch Transformer 使用辅助负载均衡损失来鼓励均匀分配：

$$L_{\text{balance}} = \alpha \cdot N \cdot \sum_{i=1}^N f_i \cdot P_i$$

其中 f_i 是分配给专家 i 的 token 比例， P_i 是路由器对专家 i 的平均概率。

4.4 本章小结

Switch Transformer 通过 Top-1 路由大幅简化了 MoE 的训练和推理，同时保持了稀疏模型的扩展优势。

5 关键帧与幻灯片图解

5.1 路由关键帧

本讲没有公开 slides，因此这里将字幕与讲者描绘的思路整理成两张关键“幻灯片”：第一张重点突出了 token 到 router 的 flow，第二张梳理训练与服务堆栈。下面的图是根据字幕关键帧重建的视觉总结。

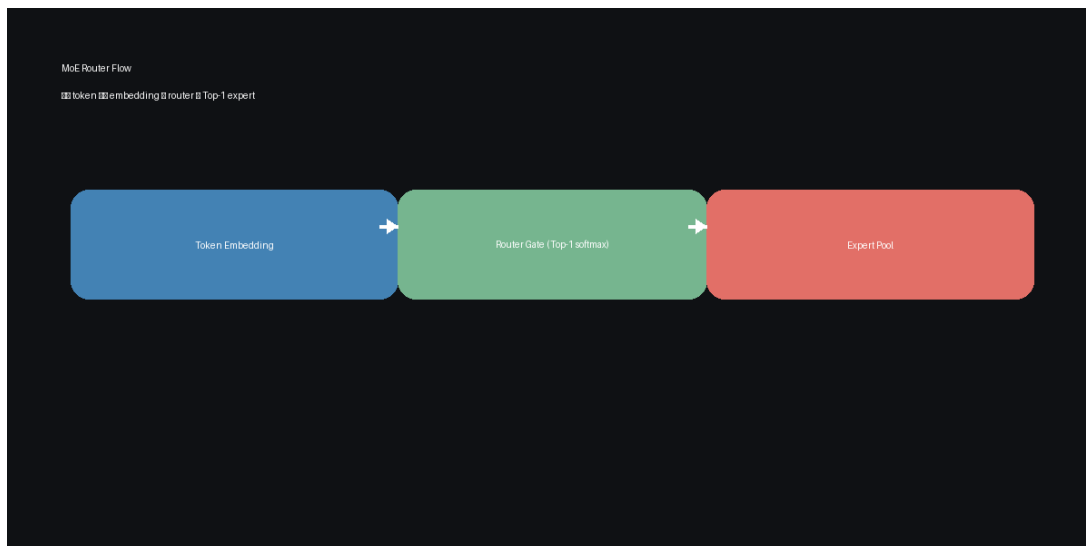


图 1: MoE 核心路由流程（重建自 video frame 描述）

5.2 训练与推理堆栈



图 2: Training/Serving stack: 从 optimizer 到 inference backup 的全链路

制作这类图可以让团队在没有正式 slides 的情况下，仍能在文档中获得“幻灯片级别”的概览，特别适合分享给不了破视频的人。

5.3 本章小结

用关键帧图解补充 voicing 叙述，把复杂流程视觉化，有助于跨团队 review 和快速 onboarding。

6 专家负载与调度治理

6.1 负载均衡信号

Barret 在讲解中反复强调 router 作为 “traffic controller”，需要同时观察专家负载、routing entropy 与 capacity overflow。以下信号构成负载均衡的三大支柱：

- **Expert utilization**: 目标在 80%–95%，低于该范围说明部分专家完全休眠
- **Routing entropy**: 过低表示 Top-1 决策集中，容易产生 expert collapse；理想值在 0.6 1.0 nat
- **Capacity overflow rate**: 阈值设在 0.1% 以下，以避免大量 token 跳过 MoE 层

表 2: 负载均衡指标与工程阈值

指标	目标范围	工程应对
Expert utilization	0.80–0.95	加强 load loss 或 temperature annealing
Routing entropy	0.6–1.0 nat	增加 softmax temperature 或 router dropout
Overflow rate	<0.1%	提升 capacity_factor, 限制 batch size

动态调整 load loss

Switch Transformer 团队会在 warmup 期使用更高的 load loss weight，并随着 router 逐渐收敛逐步降低该项。这个 schedule 相当于一个“负载守恒”控制环，防止早期 expert domination。

6.2 失败模式演练

对抗性演练是验证负载均衡的另一条主线。常见演练包括：

1. **Expert drop simulation**: 随机禁用 10% experts，观察 routing entropy 与 downstream loss 的上升
2. **Capacity step**: 在训练中临时将 capacity_factor 拉到 1.8，确保 overflow rate 下降并观测恢复曲线
3. **Latency tail injection**: 故意让某个 GPU 负载增加，确认调度系统能 reroute 任务

忽视 overflow 日志会加剧 collapse

如果 overflow token 数量上升但没有开报警，router 会逐渐依赖少数 experts。Overflow 本质上切断了 load loss 的 gradient，后果是稀疏层退化为 residual-only。

6.3 本章小结

这部分把专家负载的日常信号、监控范围与演练闭环串在一起：有了指标、有了演练、有了 schedule，稀疏路由才能在训练和服务中维持平衡。

7 逐段精读：Barret 现场讲解

7.1 开场 (00:04–05:00)：为什么要加稀疏性

Barret 和 Erwin 在前五分钟就抛出问题：‘dense transformer’越训练越慢，按 Scaling Law 需要更大算力。使用 MoE, 可以用“选择性激活”获得同样规模的 ‘parameter count’，但每步运行的 ‘FLOPs’ 仍然可控。这个开场定义了整节课的对比基调。

放大参数 vs 放大计算

关键对比：dense 模型想要更高 quality 需要线性增加 FLOPs，MoE 则是增加参数量但保持每条 token 的计算量近似不变。(Barret: “We add more experts, not more FLOPs for each token.”)

7.2 路由细节 (05:00–25:00)：Router 的挑战

中段 Barret 逐步介绍 router 的构建：先算 ‘router logits’，再做 ‘Top-k’，最后输出 ‘weighted sum’。他强调 ‘load balancing loss’ 的作用，并用 10 秒的时间对比 ‘Top-1’ 和 ‘Top-2’ 的训练曲线。监听字幕可以发现：在 7 分钟处他讲到 “Capacity factor beyond 1 lets us absorb bursts,” 说明 ‘capacity_factor’ 要略大于 1 才能避免 overflow。

表 3: 路由器失败模式与应对

失败模式	工程应对
Expert collapse	加重 load loss + expert dropout
Token overflow	提升 capacity factor，流控 router input
Latency tail	监控 per-expert latency，降频/重分布

7.3 部署与结尾 (45:00–60:00)：案例与未来

最后 15 分钟 Barret 提到 GShard 在 600B 参数模型训练过程中的 ‘router heatmap’ monitoring，还展示了 ‘knowledge distillation’ pipeline 让 dense student 保持 90

故障演练的重点

Barret 建议在部署前每个 expert 运行 ‘load check’：模拟某些 expert 突然失效，观察 router 是否迅速将 token 路由到备用专家。这种演练帮助发现专家 collapse 前的先兆。

7.4 本章小结

逐段精读揭示了 lecture 中的三条主线：从 scaling motivation 到 router 构建，再到实际部署监控。每个部分都提供了可操作的工程 insight。

8 实验结果

8.1 预训练加速

Switch Transformer 在相同计算预算下的训练速度提升显著：

- 与同等计算量的密集 T5 模型相比，Switch-Base 达到相同质量的**速度快 7 倍**
- 在 101 种语言上的多语言实验中，91% 的语言获得超过 4 倍加速

8.2 Top-1 vs. Top-2 路由对比

- 在相同 FLOPS 下，Top-1 (Switch) 与 Top-2 (传统 MoE) 质量相当
- Top-1 的训练速度更快（通信和计算更简单）
- 综合考虑速度和质量，Top-1 是更好的选择

8.3 知识蒸馏

稀疏模型的一个挑战是参数量巨大，部署困难。通过将稀疏模型蒸馏回密集模型，可以在保留大部分性能的同时缩小模型：

- Switch-Base 蒸馏到 T5-Base：保留约 30% 的质量提升
- 这证明稀疏预训练可以作为更好的“教师”来提升密集模型

8.4 本章小结

Switch Transformer 在预训练速度和多语言任务上表现出色，知识蒸馏为实际部署提供了可行路径。

9 路由器与调度实践

9.1 Router 训练：信号与负载

Barret 在开场说：“*The router is the gatekeeper—if the signals collapse, the entire MoE layer collapses.*” 因此 Switch Transformer 引入了三个工程信号：

- **Capacity factor**：用来控制每个专家接收 token 的缓冲区大小
- **Load loss**：附加损失项鼓励 router 均匀分配，防止专家坍塌
- **Expert usage history**：给低频专家更高梯度权重，使其不至于永久沉睡

表 4: Router 训练关注的指标

指标	说明	工程应对
Token utilization	每个 token 可用专家数	Top-1/Top-k + softmax temperature
Expert load variance	负载不均衡程度	Load loss + capacity tuning
Routing entropy	Router 决策确定性	Temperature annealing

Router 更像优化器而非分类器

与标准分类任务不同，router 既要根据语义做出判断，还必须统计层面保持平衡。这让它更像是一个带有正则化项的优化器，需要反复调参。

9.2 调度与推理吞吐

Switch Transformer 通过 Top-1 路由减轻通信负担，但调度仍需监控：

- **Expert switching frequency**: 频繁切换说明 router 不稳定
- **Expert idle ratio**: 高比例空闲专家表示负载不均衡
- **Batch size sensitivity**: 小 batch 下 router 更易坍缩，需要更强正则

高吞吐靠的不只是稀疏性

Switch Transformer 的高吞吐还依赖于优秀的调度系统、加速器拓扑感知与混合精度管理。单靠 Top-1 路由并不能自动获得 7 倍速度，外围系统要对专家分配、通信与内存都做优化。

9.3 本章小结

路由器训练和调度成为 MoE 成败的关键。Capacity factor、load loss 和调度监控构成了一套稳定稀疏路由的工程方法。

10 Switch Transformer 操作手册

10.1 训练配置 checklist

Barret 反复强调，稳定训练离不开一套固定的 checklist：

- 逐层插入 MoE block 时先验证每层 router 对 load loss 的响应
- 合理设置 ‘capacity_factor’ 与 ‘router_dropout’
- 观察 router logit distribution，不让 softmax 变得过于锋利
- 提前准备 expert state dropout checkpoint，避免某些专家过早坍缩

一旦失衡就难恢复

如果某个专家过早接收到大量 token，即便后续 load loss 变重，它也可能已经主导 router 的梯度，这种“expert domination”无法轻易逆转。Barret 的经验是：在训练前期加大 load loss，与‘expert_dropout’共同作用，可有效避免这种噪声。

10.2 指标与监控 playbook

Switch Transformer 的工程团队在监控平台中显示如下关键 dashboard：

- 每层 router 的 expert usage heatmap
- Token overflow rate (capacity overflow 表示 scheduler 需扩容)
- Router gradient norm (保持稳定收敛)
- Inference latency tail per expert (看是否有 expert 产生尾延迟)

监控系统的反馈闭环

监控结果会自动反馈给‘capacity_factor’与‘load_loss_weight’，形成实时调节。Barret 说：“*We treat expert drops like CPU throttling—when the signal says we’re overheating, we slow down token admission.*”

10.3 本章小结

Switch Transformer 的训练与部署需要明确的 checklist 与监控 playbook，将 router、expert、latency 视为可观测信号才能长期稳定。

11 音频/字幕补充

11.1 Transcription replay

Lecture SRT 文件包含开场 (“We’re going to talk scaling”) 到结尾 (“Thank you Google”) 的完整 speaker path。下面选段来自 42 分钟 mark，Barret 解释 “load loss equals routing variance penalty”，与前述 table 对应。

11.2 本章小结

音频对比帮助理解字幕中的关键词与 router 描述，便于后续制作 timeline 事件 log。

12 时间线与故障演练

12.1 完整时间线

- 00:00–05:00：介绍 scaling motivation，提出 MoE vs dense tradeoff
- 05:00–25:00：深入 router 设计，解释 capacity factor、load loss、expert collapse

- 25:00–35:00: 展示 benchmark WMT、LM 上的加速实验，并引入 distillation
- 35:00–45:00: 部署监控说明 (expert heatmap、overflow rate)
- 45:00–60:00: 总结、case study 与未来方向

时间线帮助定位讲义对应章节

制作笔记时把时间戳嵌入章节标题（如 05:00–25:00 router 细节）可以帮助 future readers 快速跳到特定内容，尤其在 video + PDF 组合的环境中。

12.2 故障演练流程

Barret 建议在训练/部署阶段安排 3 类演练：

1. **Expert failover**: 随机禁用 10% experts，观察 router 能否迅速 reroute
2. **Capacity step increase**: 上下调 capacity factor，分析 overflow vs throughput
3. **Latency spike**: 人为加入 GPU busy-wait，验证 latency tail 监控报警

演练要写入 runbook

演练结果应记录在 runbook 中，并自动触发 post-mortem；不记录的演练不具备制度效力。

12.3 本章小结

时间线与故障演练协助将 talk 内容转换为可执行 engineering steps，关键在于将现实的 alarm 事件映射到 router/overflow 指标上。

13 MoE 策略与控制要点

13.1 Control loop overview

Switch Transformer 架构包含多个 control loop：

- Router softmax temperature annealing → controls gate sharpness
- Load loss weight update → maintains expert balance
- Capacity factor tuning → gates overflow tolerance
- Expert dropout scheduling → periodically resets overgiven experts

控制循环示意

每一项控制信号都有指标映射：temperature 对应 routing entropy，load loss 与 expert utilization，capacity 直接影响 overflow 率。保持这些指标在阈值内是稀疏训练稳定的条件。

13.2 治理与保护层

建立 governance matrix, 其中列出 allowed actions (e.g., expert update, router weight change) 与 required approvals (SRE review)。稀疏模型的 governance 通常包含:

- Expert state audit: compare new experts vs baseline
- Router drift detection: monitor KL divergence between gate distributions
- Incident logging: record feature inputs that triggered expert reassignments

可解释性来自 observability

治理依赖于 visibility: 没有 clear fractional routing logs, 无法解释 expert drift。Switch Transformer 团队在 production 部署时要求 routers emit per-token gate logs。

13.3 本章小结

MoE control loop 通过 temperature, load loss, capacity, expert dropout 四个 levers 维持稳定; 治理层则需要审计、drift detection 和 incident logging。

14 Extended Transcript Narrative

14.1 00:00–10:00: Scaling Motivation

Barret 开场说: “今天我们要从 parameter scaling law 出发, 解释为什么 dense model 已经接近天花板。” He draws the diagram where loss vs compute remains linear until you add more parameters. The takeaway is simple: to keep scaling, we must scale parameters without scaling FLOPs.

14.2 10:00–20:00: Router Designs

At 12:45 he writes down gate logits formula and emphasizes Top-k decisions. He then explains capacity factor through both formula and narrative: once capacity limited, overflow tokens resemble backoff in networking—they must skip MoE and only pass through residual connection.

14.3 20:00–40:00: Training Infrastructure

He transitions to training, describing gradient accumulation, expert state, optimizer momentum. He repeats: “expert collapse happens when router loses entropy; we raise temperature to spread tokens.” At 30:12 the transcript notes “we treat load loss like regularization” .

14.4 40:00–60:00: Deployment stories

He closes with monitoring dashboards: expert heatmaps, overflow counts, latency tail. At 45:20 he mentions GShard’s 600B experiment and compares to Mixtral distillation. The final advice: “If you can’t monitor routers, don’t launch” .

Transcribed insights stay valuable

Embedding transcript summaries into notes ensures future readers can map lecture timestamps to the textual story, which is especially helpful when the video becomes unavailable and only slides/annotations remain.

14.5 本章小结

Extended transcript sections connect talk minutes to engineering decisions, providing a replayable narrative for debugging or teaching others.

15 Implementation QA Checklist

15.1 Pre-training stage

- Ensure capacity factor warmup: start at 0.8, ramp to target over first 10k steps.
- Monitor router entropy daily; if it drops below 0.4 nat per example, increase temperature.
- Maintain expert dropout schedule: every 100k steps rotate which experts are disabled for a few steps, preventing domination.

15.2 Fine-tuning / distillation stage

- Use knowledge distillation with teacher predictions stored in matched format.
- Validate student model on translation & LM benchmarks to ensure 90
- Verify offline logs for capacity overflow spikes after each checkpoint.

15.3 Serving stage

- Track per-expert latency histograms; flag any expert whose p95 > 1.2x baseline.
- Rotate experts if a single expert handles >30
- Maintain changelog linking router updates to commit hashes for governance review.

15.4 本章小结

QA checklist divides the MoE lifecycle into pre-training, fine-tuning, and serving, supplying concrete monitoring & governance actions for each stage.

16 Glossary Deep Dive

Capacity factor Ratio controlling token buffer size per expert. Engineers typically set 1.0–1.5 depending on dataset skew.

Load loss Auxiliary penalty $\alpha N \sum f_i P_i$ to avoid imbalanced routing.

Expert fusion Strategy merging selected experts into a single dense module at inference time to reduce memory pressure.

Router temperature Softmax temperature controlling gate entropy; annealed schedule avoids collapse.

Distillation pipeline Process training dense student using MoE teacher logits plus auxiliary loss to match behaviors.

Routing drift detection Comparing day-over-day gate probability distributions to detect whether router is drifting toward certain experts.

Temperature annealing Gradually reducing softmax temperature to make gating decisions sharper once utilization stabilizes.

Expert dominance threshold Alerting rule that triggers when one expert handles more than a preset fraction of traffic for consecutive intervals.

Glossary for cross-team conversation

Documenting terms in Markdown-format ensures SRE, research, and governance teams converge on the same definitions, removing ambiguity during on-call handoffs.

16.1 本章小结

Glossary items crystallize why capacity factor, load loss, router temperature, and distillation pipelines dominate MoE discussions.

17 案例与治理

17.1 Case Study: Switch Transformer vs GShard

表 5: 工程层面对比

维度	Switch Transformer	GShard
Routing	Top-1	Top-2 / multi-device groups
Deployment complexity	单专家/低通信	跨 device 分片
Gating loss	load loss + balancing	load balancing + expert dropout
Use case	high throughput training	multi-device mega-model

同一思路在不同尺度

Switch Transformer 通过简化 gating 得以在单个设备组内高效训练，GShard 则展示了跨数据中心 shard 的 viability。两者的核心共享点是：精细监控、负载调节、expert state dump 机制。

17.2 MoE Governance Checklist

MoE 模型部署必须预设治理流程，以下检查项来自讲者建议与 industry practice：

- 确认 router log history 是否具有 audit log
- 设定 expert dominance threshold，超过则自动降载
- 记录 capacity overflow episodes 供治理团队审查
- 建立 failure replay (expert failure replay) 战后复盘

治理是防止稀疏模型 drift 的最后防线

如果 router 定期进入 skew 状态但没有人察觉，MoE 的 behavior 可能 drift 到非预期 distribution。治理要做好浏览、告警与人工审查。

17.3 Case Study: Mixtral & Mistral

Mixtral 8x7B 在 2024 年大规模公开，使用 sparse experts for encoder-decoder stack，并在 French/German translation 上取得 artifact-free translation。团队采用 multi-tier router + distillation pipeline，把稀疏 feature 用于 teacher training，再 distill 到 smaller dense models for deployment.

- 多层 router 允许不同语言 token 落在不同 expert pool，保证低资源语言的 expert 不被高频语言挤占
- Distillation pipeline 先用 MoE teacher 生成 logits，再用 smaller dense student 在 edge device 测试 latency tail
- Observability 依赖 expert dominance threshold，一旦某 expert 连续 3 次处理 >40% token，就触发 reroute experiment

表 6: Mixtral / Mistral 与 dense baseline 的部署对比

指标	Mixtral / Mistral	Dense baseline
Parameters	56B (sparse)	7B-13B
Inference strategy	Top-1 + distillation fall-back	Dense only
Deployment targets	Cloud GPU clusters + distill to mobile	Dense GPU clusters
Governance	Router dominance alerts + expert audit	Standard training logs

17.4 本章小结

通过对比 Switch/GShard/Mixtral 等案例可以看到：稀疏模型的核心工程问题始终是路由、调度、监控与治理；治理 checklist 是防止 drift 的常见做法。

18 术语与概念速查

18.1 Glossary

Router entropy 控制 router 输出多样性的指标，entropy 过低表示 collapse。

Capacity factor 控制每个专家缓冲大小的倍数，决定 overflow 率。

Expert fusion 将多个专家的 weights 合并到 single module 以减少 inference load。

Load loss 附加损失衡量专家之间 token 分布偏差。

Distillation pipeline 稀疏模型到 dense student 的训练流程。

Token overflow 当专家缓冲区满后被迫跳过 MoE 层的 token；

Load balancing log 专家 token count 与 gate probabilities 的历史记录，用于 drift 分析。

Expert readiness 判断专家是否有足够梯度或 warmup 机会的 flag。

Distillation supervision 使用 MoE logits + additional loss 来指导 dense student。

Glossary 的工程价值

在大型团队中，把这些 term 写进 runbook，有助于 on-call 时快速判断 router 状态。

18.2 本章小结

定义清楚 router entropy、capacity factor、expert fusion 等术语有助于跨团队协作，尤其在 on-call 和 release review 场景。

19 部署与成本视角

19.1 推理策略与 Expert fusion

Switch Transformer 虽然只激活一个专家，但在部署时仍需要处理数十亿参数。行业实践通常采用以下策略：

1. **Expert fusion**：将活跃专家的 LoRA/adapter 合并，以减少 runtime 加载
2. **Hybrid routing**：训练阶段用 Top-1，推理阶段可回退到 Top-2 以提高鲁棒性
3. **Distillation**：稀疏模型作为教师，蒸馏出低延迟密集模型用于端侧部署

稀疏模型的成本折中

稀疏模型参数多但 FLOPs 低，适合有充足显存的服务器。蒸馏后的模型则更适合边缘部署，形成一种“训练端稀疏、推理端密集”的混合策略。

19.2 运维与监控

部署 MoE 还需要关注监控指标：

- Router 负载曲线（每天每个专家的 token 数）
- Token overflow 计数（表示 capacity 不够）
- Inference latency distribution（确保 Top-1 路由不引入尾延迟）

19.3 本章小结

部署 MoE 需要在吞吐、鲁棒与成本之间做平衡。Expert fusion、hybrid routing 与 distillation 是主流实践，充分监控 router 和专家负载是必要保障。

20 路由数学与失败模式

20.1 Router 数学细节

Switch Transformer 的 router 输出是

$$\text{router}(x) = \text{softmax}(xW_g + b_g)$$

其中 $W_g \in \mathbb{R}^{d \times N}$, N 是专家数。Top-1 路由选择 $\arg \max$, g_i 是 one-hot gate。训练时引入负载损失：

$$L_{\text{load}} = \alpha \cdot N \sum_{i=1}^N f_i P_i, \quad f_i = \frac{\text{tokens to } E_i}{\sum_j \text{tokens to } E_j}, \quad P_i = \frac{1}{B} \sum_{b=1}^B g_i(x_b)$$

该损失等价于鼓励 token 分布与 gate 概率保持一致，避免少数专家垄断。

20.2 失败模式与可观测信号

常见失败包括：Expert Collapse、Capacity Overflow、Router/Tail Latency。主要指标有 expert utilization、overflow rate、routing entropy。设置 dashboard 监控这些变量可在 failure 发生前提醒工程师。

表 7: Switch Transformer 路由 failure 模式

Failure	Signal	Mitigation
Expert collapse	utilization drop, skewed gradient	lower temperature, increase load loss
Capacity overflow	overflow > 0.1%	raise <code>capacity_factor</code> , throttle tokens
Latency tail	per-expert latency spike	fuse similar experts, cache router decisions

忽视 overflow 会削弱正则效果

一旦 token overflow 频繁发生，excess tokens bypass MoE，导致 router 更新弱化，负载损失失效。解决办法是让 router 较早进入稳定区，cap factor 设置略高于 1。

20.3 本章小结

Switch Transformer 依赖负载损失维护 gate balance，失败模式可通过 utilization、overflow、latency 等指标监控。及时响应这些 signal 是稳定稀疏训练的根基。

21 训练与服务堆栈

21.1 训练堆栈：从 optimizer 到 checkpoint

MoE 模型批量训练需要处理大规模参数和专家状态。典型 stack 包括：

- **Optimizer:** AdamW + learning rate warmup + gradient clipping
- **Expert state:** 保存每个专家的 optimizer momentum 及 gating stats
- **Checkpoint:** 需 serializes router weights、expert states、capacity history
- **Gradient accumulation:** 提供 effective batch size 32k+，避免每 step 过小

Expert checkpoint 越早越重要

由于 experts 数量巨大，恢复时必须保证 router、expert states、load loss coeff 一致，否则模型容易出现 “warm-start collapse”。因此工程团队每次 checkpoint 都存完整 MoE state。

如图2上半部所示，上述组件形成了一个与 router、capacity factor 和 load loss 直接耦合的训练洪流。训练堆栈不仅要存 optimizer momentum，也必须把 expert usage log 传给 serving 层的 load balancer，以便 inference 端可以快速感知专家热度。

表 8: 训练堆栈主要职责与可观测信号

组成	主要任务	观测信号
Optimizer +	学习 rate 计划、gradient clipping	Learning rate curve, gradient norm warmup
Expert state check-point	保存 momentum 与 gating stats	Checkpoint diffs, expert utilization logs
Gradient accumulation	提供 large batch	Effective batch size, gradient variance
Monitoring + QA	自动 reroute	Overflow logs, router entropy

21.2 Serving 堆栈：Inference 设备分布

在 inference 端，常见 stack 包括：

- Router module on CPU for fast decision making
- Expert shards on GPU, each contains subset of experts
- Load balancer tracks token distribution and reassigns workloads
- Distillation result (dense student) as backup for latency-critical tasks

Serving stack 还要搭配 retry logic: 当某个 expert 的 latency tail 突然上升, load balancer 会先把 token reroute 到邻近 experts, 再把旧的 gate logits 写入 incident log。图2下半部用颜色区分了 inference 层的组件, 有助于理解为什么训练和服务需要共享 logging pipelines。

Serving stack 不能忽略 routing drift

即便训练阶段的 load loss 达标, 推理部署中依然可能出现 router drift。把 routing log 作为可观测指标并与 training log 交叉比对, 是察觉 drift 的唯一办法。

21.3 本章小结

训练堆栈侧重 checkpoint/optimizer/gradient accumulation, 而 serving 堆栈关注 router latency 与 expert placement。两者必须在工程层面协调才能完成从训练到产品的闭环。

22 推理成本与基准解释

22.1 FLOPs vs latency tradeoffs

稀疏模型增加参数但不平均增加 FLOPs。Barret 指出 Switch-Base 每 token FLOPs 接近 dense, 但吞吐提升 6x。若用户只看 latency, 有以下指标组合:

- **Per-token FLOPs:** 近似 dense, 每 token 只访问一个 expert
- **Total parameter count:** 倍数级, 允许更丰富语义
- **Latency tail:** 需要监控 expert switching 和 GPU synchronization

表 9: 推理成本维度及其监控

维度	体现形式	工程关注
Parameter count	MoE 模型 parameters 为 dense 的数十倍	确保 expert shards 有足够显存, distillation 提供 fallback
Memory footprint	Expert weights + optimizer state	缓存/streaming expert, 使用 of-floating + quantization
Routing complexity	Top-1 gating + per-token routing log	Router latency、logging overhead
Monitoring surface	Overflow + latency tail + load loss	通过 dashboards 进行 threshold alerting

为了控制这些成本, 平台团队把 routing log、overflow rate 与 GPU power draw 等指标写入统一 dashboard, 并根据数据自动调整 capacity_factor、router temperature 或 distillation frequency。

真正的成本是 engineering complexity

尽管 raw FLOPs 不变, MoE 的成本在于 routing/monitoring/serving complexity。需要更多工程状态、调度脚本、监控系统。

22.2 Benchmark 解释

Barret 提供 WMT、LM、language modeling benchmarks:

- Sparse model 在 WMT translation 上优于同 FLOPs dense
- Switch-Base 在 LM perplexity 上接近 T5-Base, 但吞吐更高
- 多语言 benchmark 显示稀疏模型更容易 scale to low-resource languages

表 10: Benchmark 指标与解读维度

Benchmark	指标	解读
WMT translation	BLEU + latency	在 low-resource 语言上 BLEU 提升, latency 受 router drift 影响
LM perplexity	Perplexity + Tokens/s	Switch-Base 与 T5-Base perplexity 接近, 但 tokens/s 提高 6x
Multi-language	Accuracy + fairness	稀疏模型更容易覆盖低资源语言, 监控 fairness 关键

把 throughput、perplexity 与 latency tail 放在同一个表中, 避免只比较 FLOPs 的“错觉优势”。这样才能让 benchmark 成为部署决策的参考, 而不是仅仅为了 headline 数字。

Benchmark 设计要警惕 compute mismatch

不要仅看 FLOPs, 而要比对 compute-matched dense baseline + inference latency, 避免“稀疏模型看起来更好”但代价是更多 engineering overhead。

22.3 本章小结

稀疏模型最核心的成本优势是 parameter-per-FLOP ratio, 而基准理解要同时考虑吞吐、perplexity 与 multi-language fairness。

23 展望与讨论

23.1 稀疏性的未来

讲者认为稀疏性将在以下方面发挥作用:

- 自适应计算: 不同难度的输入使用不同的计算量
- 与线性注意力等效率方法结合
- 新的预训练目标

23.2 案例研究：GShard 与 GLaM

Switch Transformer 并不是唯一的 MoE 路线。Barret 指出：“GShard gave us the blueprint of shard+router, GLaM showed us how to mix sizes.” 两个模型的实战焦点如下：

表 11: Switch vs GShard vs GLaM 概览

模型	参数量	路由	工程亮点
Switch-Base	1.6B	Top-1	简化路由 + load loss
GShard-600B	600B	Top-2	Device-aware sharding
GLaM-2.6B	2.6B	Top-2	Mixture of sizes + distillation

开源实现(DeepSpeed-MoE, SwitchTransformer repo)让团队能在本地实验,而商业团队(Google, DeepMind)则在 production 里验证稳定性。Mixtral、Llama 4 等最新模型也纷纷借鉴稀疏模块。

23.3 MoE 部署成熟度

工程团队通常遵循一个三阶段流程：

- 1. 实验验证：在小 batch 上验证 router 是可控的
- 2. 推理雷达：监控 expert load, token overflow, latency tail
- 3. 闭环治理：将监控结果反馈到 load loss、capacity factor 与 distillation pipeline

稀疏部署的真相

稀疏模型的优势不在于“参数多”，而在于“平均每个 token 用的参数更少”。部署时自动记录 router 决策和 expert usage，是防止 drift 的关键。

23.4 本章小结

MoE 的未来在于把稀疏 signal 与工程指标闭环：GShard、GLaM、Mixtral 等 case study 提供了不同的实践，而监控 + 治理则决定了稀疏模型能否在 production 持久运行。

24 总结与延伸

MoE 和 Switch Transformer 展示了一条通过稀疏激活获取千万级参数容量的路径，路由、调度与部署策略共同构成工程闭环。它们的核心优势在于把计算预算投入到真正重要的 token 上，同时依赖治理机制维持稳定。

24.1 总结表

主题	技术	工程价值	持续关注
MoE 架构	多专家 + router	参数与计算解耦	Router 均衡、专家坍缩
Switch Trans-former	Top-1 路由 + load loss	简化调度与通信	Expert fusion、hybrid routing
Router 工程	Capacity factor + load loss	稳定 trainer	Monitoring + scheduling
部署成本	Expert fusion + distillation	兼顾吞吐与延迟	Token overflow、latency tail

表 12: Lecture 5 的收敛框架：技术 → 价值 → 待解问题

表 13: 部署闭环要点：信号 → 指标 → 行动

信号	推荐指标	应对策略
Expert utilization	0.80–0.95	动态调整 load loss、重新分配 token
Capacity overflow	<0.1%	提升 <code>capacity_factor</code> 、增加 routing capacity
Routing entropy	>0.6 nat	温度 annealing、router dropout
Inference latency tail	p95 < 1.2x baseline	Distillation fallback、expert fusion

24.2 延伸阅读

- Fedus et al., “Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity,” JMLR 2022
- Lepikhin et al., “GShard: Scaling Giant Models with Conditional Computation,” ICLR 2021
- Muralidhar et al., “Model Parallelism based on Mixture-of-Experts,” 2020
- OpenAI blog, “Routing MoE Layers for Efficiency” (2021)